



Creación de un *pipeline* de datos con Kedro

¿Qué es un *pipeline* en Kedro?

En el corazón de Kedro se encuentran los ***pipelines*** y los **nodos** como componentes fundamentales que permiten construir flujos de datos complejos pero organizados y modulares.

Este enfoque ofrece múltiples beneficios, como los siguientes:

- **Estructura consistente:** todos los proyectos siguen el mismo patrón organizacional.
- **Separación clara:** datos, código y configuración se mantienen separados.
- **Versionado:** los datos intermedios y de salida, así como los modelos pueden ser versionados para reproducibilidad.
- **Portabilidad:** los proyectos pueden moverse entre entornos y de prototipo a producción con cambios mínimos.
- **Colaboración:** múltiples personas pueden trabajar en diferentes partes del mismo proyecto.

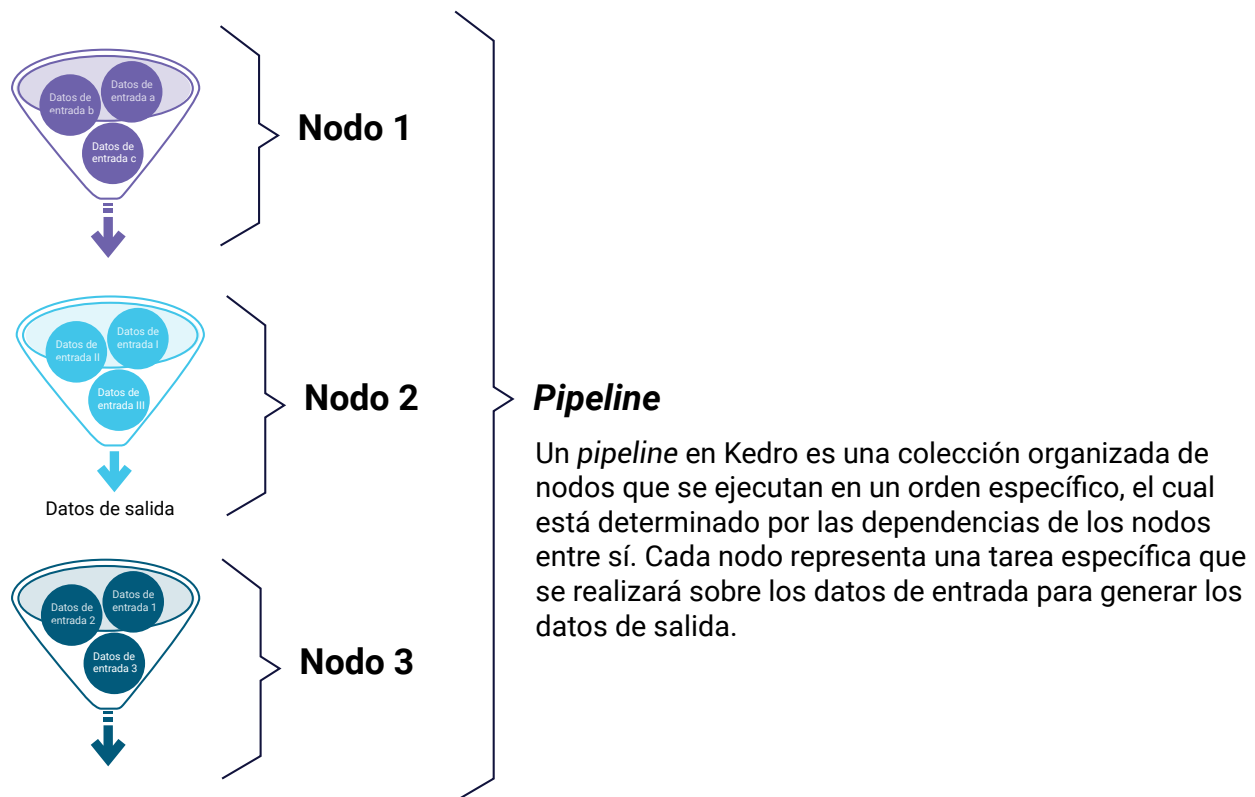


Figura 1. Estructura de un *pipeline* en Kedro.

A su vez, cada *pipeline* puede representar un flujo de trabajo completo, o bien, una parte modular de un flujo más grande.

Es importante destacar que **el *pipeline* determina el orden de ejecución de los nodos resolviendo dependencias** y no necesariamente ejecuta los nodos en el orden en que se programan. Estas dependencias se establecen automáticamente a partir de las entradas y salidas de cada nodo y siempre en una dirección específica. En esencia, un *pipeline* es un **grafo acíclico dirigido**.

Comprender qué es un **grafo acíclico dirigido (DAG, por sus siglas en inglés: *Directed Acyclic Graph*)** es fundamental, ya que se trata de una estructura matemática y de datos que se compone de:

1. **Nodos** (también llamados vértices): representan entidades o elementos.
2. **Aristas dirigidas**: conexiones entre los nodos que tienen una dirección específica, generalmente representadas con flechas.
3. **Sin ciclos**: no es posible recorrer el grafo siguiendo las direcciones de las aristas y volver al punto de partida.

Las **características fundamentales** de un grafo acíclico dirigido son:

- **Direccionalidad**: las conexiones entre nodos tienen una dirección única ($A \rightarrow B$ no implica $B \rightarrow A$).
- **Acíclico**: no contiene ciclos, es decir, no hay forma de seguir las aristas y volver al nodo de partida.
- **Ordenamiento topológico**: es posible ordenar los nodos de tal manera que para cada arista dirigida de U a V, U aparece antes que V en el ordenamiento.

Los grafos acíclicos dirigidos son extremadamente útiles y aparecen en muchas aplicaciones:

- **Programación:** Representación de dependencias entre tareas.
- **Planificación de proyectos:** Diagramas PERT/CPM para planificación.
- **Sistemas de archivos:** Representación de jerarquías de directorios.
- **Redes bayesianas:** Modelado de relaciones causales.
- **Flujos de trabajo:** Secuencias de operaciones con dependencias.
- **Compiladores:** Representación de orden de ejecución de operaciones.

La Figura 2 muestra un grafo acíclico dirigido simple, donde se puede observar cómo las aristas (flechas) siempre apuntan "hacia adelante" o "hacia abajo", sin posibilidad de regresar al punto de partida siguiendo las flechas.

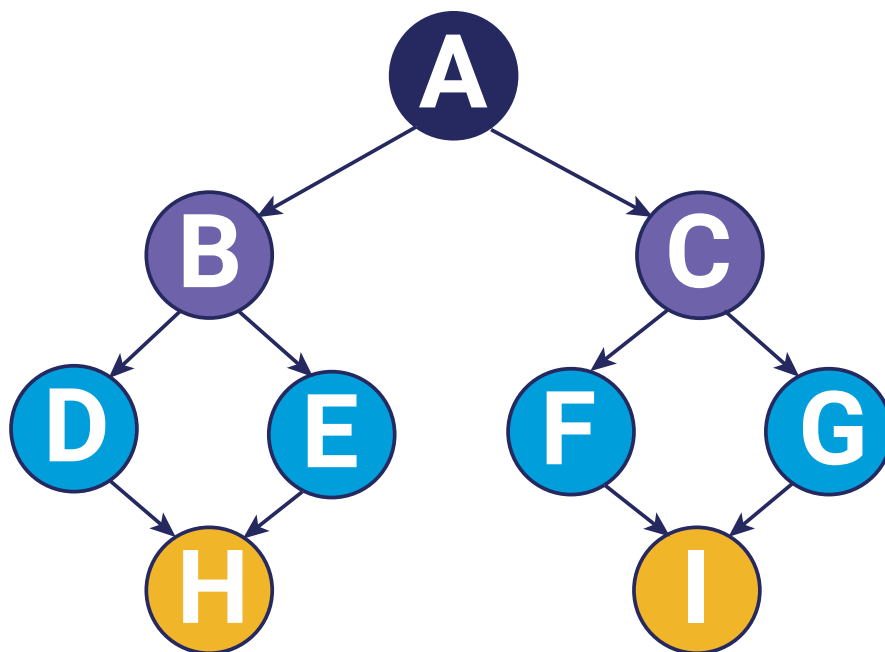


Figura 2. Grafo acíclico dirigido simple.

Para un *pipeline* de Kedro, los componentes del DAG serían:

- Los nodos son los **vértices** (puntos de procesamiento).
- Las dependencias de datos son las **aristas** (conexiones entre nodos).

Es importante destacar que esta manera de estructurar *pipelines* de datos no es exclusiva de Kedro, y se usa de manera extensiva en distintas herramientas de *pipelines*.

El proceso mediante el cual Kedro determina las dependencias entre nodos y establece el orden de ejecución es fundamental para entender cómo funcionan los *pipelines*. Esto es, un nodo B depende de un nodo A si al menos una de las entradas de B coincide con una de las salidas de A. Este grafo representa las relaciones "debe ejecutarse antes que" entre los nodos.

En la siguiente figura podemos ejemplificar cómo se determinan estas dependencias.

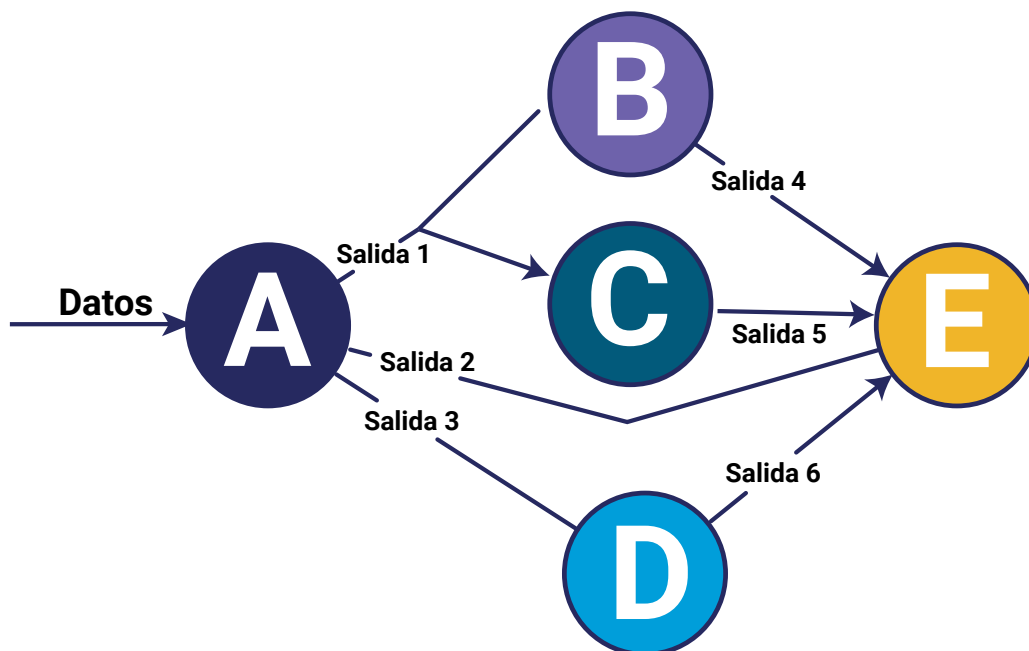


Figura 3. Ejemplo que muestra cómo se determinan las dependencias.

- El nodo A tiene tres salidas: **Salida1**, **Salida2** y **Salida3**, y una entrada: **Datos**.
- El nodo B tiene una entrada, que viene directamente de la **Salida1** del nodo A, por lo que A “debe ejecutarse antes que” B.
- El nodo C tiene una entrada, que viene directamente de la **Salida1** del nodo A, por lo que A “debe ejecutarse antes que” C.
- El nodo D tiene una entrada, que viene directamente de la **Salida3** del nodo A, por lo que A “debe ejecutarse antes que” D.
- El nodo E tiene cuatro entradas, que provienen directamente de **Salida4** del nodo B, **Salida5** del nodo C, **Salida2** del nodo A y **Salida6** del nodo D, por lo que **A, B, C y D deben ejecutarse antes que E**. El nodo E tiene una salida: **Resultado**.

En la práctica, **estas salidas y entradas se refieren a *datasets* definidos en el archivo `catalog.yml`**, es decir que, si un nodo utiliza como salida el *dataset* definido en el catálogo como **playlists**, cualquier otro nodo que utilice como entrada el mismo *dataset* será una dependencia del primero.

```
def create_pipeline(**kwargs) -> Pipeline:
    return pipeline([
        node(A, "Datos", ["Salida1", "Salida2", "Salida3"]),
        node(B, "Salida1", "Salida4"),
        node(C, "Salida1", "Salida5"),
        node(D, "Salida3", "Salida6"),
        node(E, ["Salida4", "Salida5", "Salida2", "Salida6"], "Resultado"),
    ])
```

Figura 4. Ejemplo de definición de un *pipeline* en Kedro.

Viendo un ejemplo en código, el grafo anterior se implementaría así:

De manera general, `Datos`, `Salida1`, `Salida2`, `Salida3`, `Salida4`, `Salida5`, `Salida6` y `Resultado` deben estar definidos en `catalog.yml`.

Más adelante veremos que significa cada instrucción.

IMPORTANTE: Debido a que sería imposible determinar las dependencias entre nodos, Kedro no permite utilizar un `dataset` como salida en más de un nodo. Algo como lo siguiente, causará un error:

```
def create_pipeline(**kwargs) -> Pipeline:
    return pipeline([
        node(A, "Datos", ["Salida1", "Salida2", "Salida3"]),
        node(B, "Salida1", "Salida4"),
        node(C, "Salida1", "Salida4"),
        node(D, "Salida3", "Salida6"),
        node(E, ["Salida4", "Salida5", "Salida2", "Salida6"], "Resultado"),
    ])
```

Figura 5. Error al intentar determinar las dependencias entre nodos.

IMPORTANTE: Debido a que sería imposible determinar un camino de ejecución, Kedro no permite la existencia de ciclos (ya no sería un DAG). Algo como lo siguiente, causará un error:

```
def create_pipeline(**kwargs) -> Pipeline:
    return pipeline([
        node(A, "Datos", ["Salida1", "Salida2", "Salida3"]),
        node(B, "Salida1", "Salida4"),
        node(C, "Salida1", "Salida5"),
        node(D, "Salida3", "Salida6"),
        node(E, ["Salida4", "Salida5", "Salida2", "Salida6"], "Datos")
    ])
```

Figura 6. Error al intentar determinar un camino de ejecución.

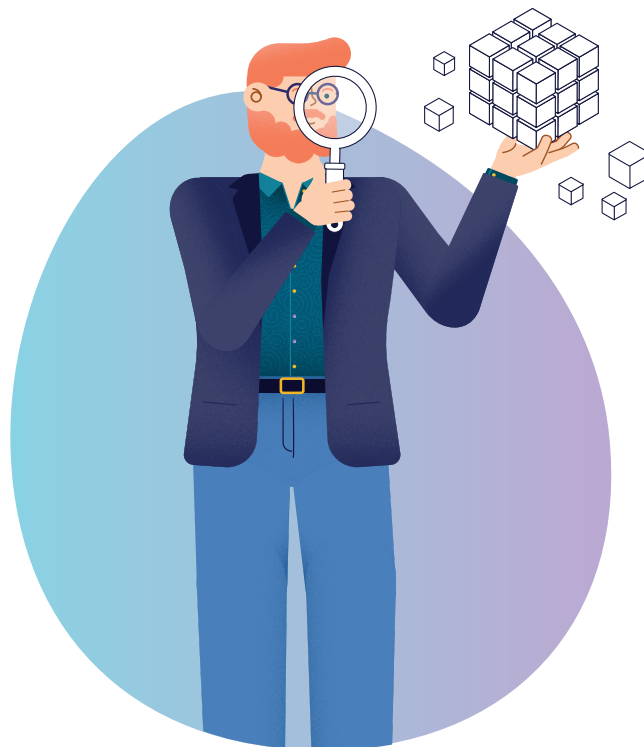
Este mecanismo de resolución automática de dependencias ofrece varias ventajas:

- **Flexibilidad en la definición:** se pueden definir los nodos en cualquier orden en el código; Kedro determinará el orden correcto de ejecución.
- **Modularidad:** se pueden añadir, eliminar o modificar nodos sin preocuparse por actualizar manualmente un orden de ejecución.
- **Paralelismo automático:** Kedro identifica de manera automática oportunidades para ejecución paralela.
- **Validación temprana:** permite detectar problemas estructurales (como ciclos) antes de la ejecución.
- **Visualización clara:** el grafo de dependencias puede visualizarse para entender mejor el flujo de datos usando Kedro Viz.

Características clave de los *pipelines* en Kedro

La clase *pipeline* de Kedro organiza las dependencias y el orden de ejecución de una colección de nodos, conectando entradas y salidas mientras mantiene el código modular.

1. **Resolución automática de dependencias:** determina el orden de ejecución basado en las entradas y salidas de cada nodo.
2. **Modularidad:** permite dividir flujos de trabajo complejos en componentes más pequeños y manejables.
3. **Reutilización:** los *pipelines* pueden combinarse y reutilizarse en diferentes contextos.
4. **Visibilidad:** proporciona una visión clara de todo el flujo de datos.
5. **Trazabilidad:** facilita el seguimiento del linaje de datos.



Definición y estructura de los nodos en Kedro

Un **nodo** en Kedro es el bloque de construcción fundamental de los *pipelines*. Representa una tarea individual o una función que realiza una transformación específica sobre un conjunto de datos. Un nodo encapsula una función de Python con parámetros para sus entradas y salidas.

Un punto a destacar es que los nodos son la unidad básica de ejecución en Kedro y deben ser:

- **Independientes:** cada nodo puede ejecutarse de forma aislada, siempre que sus entradas estén disponibles.
- **Descriptivos:** con nombres y etiquetas claras para facilitar su identificación y propósito.

Para crear un nodo, se especifica una función y los nombres de las variables de entrada y salida dentro de la función `node()`. Si deseas encontrar más información en la documentación oficial de Kedro, te dejamos el enlace en el Material adicional al finalizar esta lectura.

```
from kedro.pipeline import node
mi_nodo = node(
    func=funcion_a_ejecutar,  # La función Python que
    realizará el procesamiento
    inputs="nombre_entrada",  # Nombre(s) de la(s) entrada(s)
    outputs="nombre_salida",  # Nombre(s) de la(s) salida(s)
    name="nombre_descriptivo", # Opcional: para bitácora y
    visualización
    tags=["etiqueta1", "etiqueta2"] # Opcional: etiquetas
    para agrupar
)
```

Por ejemplo:

Analicemos los parámetros de la función `node()`

1. `func` (**obligatorio**):

- Tipo: función de Python o callable.
- Propósito: es la función que será ejecutada cuando el nodo se procese.
- Requisitos: debe ser una función sin efectos secundarios para garantizar la reproducibilidad.
- Tipos compatibles:
 - Funciones regulares definidas con `def`.
 - Funciones `lambda` (aunque no recomendadas para casos complejos).
 - Métodos de clase.
 - Funciones de librería (como `len`, `sum`, etc.).
 - Funciones de terceros.

2. `inputs` (obligatorio, excepto si la función no necesita entradas):

- Propósito: especifica de dónde provienen los datos para la función.
- Formatos aceptados:
 - `None`: indica que la función no necesita entradas.
 - `"dataset_name"`: una entrada única.
 - `["dataset1", "dataset2"]`: lista para múltiples entradas, se pasarán como argumentos posicionales.
 - `{"arg1": "dataset1", "arg2": "dataset2"}`: diccionario para argumentos con nombre, se pasarán como argumentos con nombre (`kwargs`).
 - Combinaciones de lo anterior: `["dataset1", {"arg2": "dataset2"}]`.
- Comportamiento: Kedro extraerá los datos correspondientes del catálogo antes de ejecutar la función.
- Casos especiales:
 - Uso de `"params:section.param"`: para acceder a parámetros de configuración.
 - Uso de `"parameters"`: para recibir todos los parámetros como un diccionario.

3. **outputs** (obligatorio, excepto si la función no produce salidas):

- Propósito: define dónde se almacenarán los resultados de la función.
- Formatos aceptados:
 - **None**: la función no genera salidas que deban guardarse.
 - **"dataset_name"**: una salida única.
 - **["dataset1", "dataset2"]**: lista para múltiples salidas, la función debe devolver una tupla o lista de igual longitud.
 - **{"key1": "dataset1", "key2": "dataset2"}**: diccionario, la función debe devolver un diccionario con estas claves.
- Comportamiento: Kedro almacenará los resultados en el catálogo con los nombres especificados.
- Validación: Kedro verifica que el número y formato de las salidas correspondan con lo que devuelve la función.

4. **name (opcional):**

- Propósito: proporciona un nombre descriptivo para el nodo.
- Usos:
 - Identificación en bitácoras y mensajes de error.
 - Referencia en comandos CLI (por ejemplo, **kedro run --nodes=nombre_nodo**).
 - Visualización en Kedro-Viz.
- Recomendaciones:
 - Usar nombres descriptivos que reflejen la función del nodo.
 - Mantener una convención de nomenclatura consistente.
 - Incluir sufijos como `_node` para evitar colisiones con nombres de *datasets*.
- Restricciones: el nombre debe contener solo letras, números, guiones, guiones bajos y/o puntos.

5. tags (opcional):

- Propósito: Añadir etiquetas para agrupar y ejecutar nodos selectivamente.

Formatos aceptados:

- **"etiqueta"**: una cadena para una etiqueta única.
- **["etiqueta1", "etiqueta2"]**: lista para múltiples etiquetas.
- Usos:
 - Agrupación lógica de nodos (por ejemplo, "preprocesamiento", "entrenamiento").
 - Ejecución selectiva: **kedro run --tags=entrenamiento**.
 - Organización visual en Kedro-Viz.
 - Identificación de fases del *pipeline*.

Sobre lo anterior, vemos que cada nodo está definido principalmente por sus entradas, salidas y, opcionalmente, parámetros. Esta estructura determina cómo interactúa el nodo con el flujo de datos y con otros nodos en el *pipeline* al determinar las dependencias.

Inputs (Entradas)

Las entradas de un nodo representan **los datos que la función del nodo necesita al ejecutarse**. Kedro ofrece varias sintaxis para definir entradas:

Sintaxis	Significado	Parámetros de ejemplo	Como es llamada la función
None	No hay entradas	<code>def f()</code>	<code>f()</code>
<code>'a'</code>	Entrada única	<code>def f(arg1)</code>	<code>f(a)</code>
<code>['a', 'b']</code>	Múltiples entradas	<code>def f(arg1, arg2)</code>	<code>f(a, b)</code>
<code>['a', 'b', 'c']</code>	Entradas variables	<code>def f(arg1, *args)</code>	<code>f(arg1, arg2, arg3)</code>
<code>dict(arg1='x', arg2='y')</code>	Entradas con palabras clave	<code>def f(arg1, arg2)</code>	<code>f(arg1=x, arg2=y)</code>

Tabla 1. Sintaxis de entradas.

Outputs (Salidas)

Las salidas definen los nombres de los **datasets** que serán producidos por el **nodo**. Al igual que con las entradas, hay varias sintaxis disponibles:

Sintaxis	Significado	Ejemplo de retorno
None	Sin salida	<i>Does not return</i>
<code>'a'</code>	Salida única	<code>return a</code>
<code>['a', 'b']</code>	Lista de salida	<code>return [a, b]</code>
<code>dict(key1='a', key2='b')</code>	Diccionario de salida	<code>return dict(key1=a, key2=b)</code>

Tabla 2. Sintaxis de salidas.

Parameters (Parámetros)

Los parámetros en Kedro permiten ajustar el comportamiento de los nodos sin modificar el código. Estos se definen en archivos de configuración (por ejemplo `conf/base/parameters.yml`) y se acceden en los nodos utilizando la sintaxis especial `params:nombre_parametro`.

```
model_options:
  n_estimators: 100
  max_depth: 10
  random_state: 42
preprocessing:
  test_size: 0.2
  missing_values: "mean"
```

Figura 7. Ejemplo de configuración de parámetros en `parameters.yml`.

Estos parámetros se podrían usar en nodos de la siguiente manera:

```
# Uso de parámetros como entrada
node(
    entrenar_modelo,
    inputs=["X_train", "y_train", "params:model_options"],
    outputs="modelo")

# Acceso a parámetros anidados
node(
    dividir_datos,
    inputs=["datos", "params:preprocessing.test_size"],
    outputs=["X_train", "X_test", "y_train", "y_test"])

# Acceso a todos los parámetros
node(
    pipeline_completo,
    inputs=["datos", "parameters"],
    outputs="resultados")
```

Figura 8. Uso de parámetros en nodos.

Consideraciones Importantes para la Creación de Nodos

1. Correspondencia entre entradas y firma de la función:

- El número y orden de las entradas debe corresponder con los parámetros de la función.
- Para entradas como lista `["a", "b"]`, la función debe aceptar dos parámetros posicionales.
- Para entradas como diccionario `{"x": "a", "y": "b"}`, la función debe aceptar los parámetros nombrados `x` e `y`.
- Los nombres de las entradas generalmente corresponderán a *datasets* definidos en `catalog.yml`.

2. Correspondencia entre salidas y retorno de la función:

- Para una salida única `"result"`, la función debe devolver un único valor.
- Para salidas múltiples `["a", "b"]`, la función debe devolver una tupla o lista de dos elementos.
- Para salidas tipo diccionario `{"x": "a", "y": "b"}`, la función debe devolver un diccionario con claves `x` e `y`.
- Los nombres de las salidas generalmente corresponderán a *datasets* definidos en `catalog.yml`.

3. Buenas prácticas:

- Mantén las funciones del nodo pequeñas y con un propósito único.
- Documenta claramente el propósito, entradas y salidas de cada nodo.
- Usa nombres descriptivos y significativos.
- Sigue una convención coherente para nombres y etiquetas.
- Evita efectos secundarios en las funciones del nodo.
- Considera la posibilidad de pruebas unitarias para funciones de nodos.

4. Depuración de nodos:

- Usa el parámetro **name** para identificar fácilmente el nodo en bitácoras de error.
- Implementa **logging** dentro de las funciones para seguimiento.
- Prueba las funciones del nodo independientemente antes de integrarlas al *pipeline*.

```
# Nodo simple con una entrada y una salida
nodo_limpieza = node(
    func=limpiar_datos,
    inputs="datos_crudos",
    outputs="datos_limpios",
    name="limpiar_datos_crudos")

# Nodo con múltiples entradas y una salida
nodo_fusion = node(
    func=fusionar_datos,
    inputs=["datos_clientes", "datos_productos", "datos_ventas"],
    outputs="datos_fusionados",
    name="fusionar_fuentes_datos")

# Nodo con una entrada y múltiples salidas
nodo_division = node(
    func=dividir_conjunto,
    inputs="datos_procesados",
    outputs=["conjunto_entrenamiento", "conjunto_prueba"],
    name="dividir_datos_entrenamiento_prueba")

# Nodo sin entradas (por ejemplo, para generar datos)
nodo_generador = node(
    func=generar_datos_aleatorios,
    inputs=None,
    outputs="datos_generados",
    name="generar_muestra_aleatoria")

# Nodo sin salidas (por ejemplo, para visualización o logging)
nodo_reporte = node(
    func=generar_reporte,
    inputs="resultados_modelo",
    outputs=None,
    name="generar_reporte_evaluacion")
```

Figura 9. Descripción de nodos.

Conexión entre nodos y ejecución del *pipeline*

La conexión entre nodos en un *pipeline* se establece a través de los nombres de las entradas y salidas. **Cuando la salida de un nodo tiene el mismo nombre que la entrada de otro nodo, Kedro establece automáticamente una dependencia entre ellos.** Revisemos un ejemplo:

```
# Nodo 1: Limpia los datos crudos
nodo_limpieza = node(
    func=limpiar_datos,
    inputs="datos_crudos",
    outputs="datos_limpios",
    name="limpiar_datos")

# Nodo 2: Procesa los datos limpios
nodo_procesamiento = node(
    func=procesar_datos,
    inputs="datos_limpios",  # Coincide con la salida del
nodo anterior
    outputs="datos_procesados",
    name="procesar_datos")

# Nodo 3: Divide los datos procesados en conjuntos de
entrenamiento y prueba
nodo_division = node(
    func=dividir_datos,
    inputs="datos_procesados",  # Coincide con la salida del
nodo anterior
    outputs=["X_train", "X_test", "y_train", "y_test"],
    name="dividir_datos")
```

```

# Nodo 4: Entrena un modelo con los datos de entrenamiento
nodo_entrenamiento = node(
    func=entrenar_modelo,
    inputs=["X_train", "y_train", "params:model_options"],
    # Coincide con parte de la salida del nodo anterior
    outputs="modelo",
    name="entrenar_modelo")

# Nodo 5: Evalúa el modelo con los datos de prueba
nodo_evaluacion = node(
    func=evaluar_modelo,
    inputs=["modelo", "X_test", "y_test"], # Combina salidas
de nodos anteriores
    outputs="metricas_evaluacion",
    name="evaluar_modelo")

```

Una vez que se definen los nodos, tenemos que definir el *pipeline*, como se muestra a continuación:

```

pipeline_completo = pipeline([
    nodo_limpieza,
    nodo_procesamiento,
    nodo_division,
    nodo_entrenamiento,
    nodo_evaluacion
])

```

En este ejemplo, Kedro establecerá automáticamente el siguiente orden de ejecución basado en las dependencias de datos, sin importar el orden en el que se definan dentro de la función `pipeline()` :

1. `nodo_limpieza`
2. `nodo_procesamiento`
3. `nodo_division`
4. `nodo_entrenamiento`
5. `nodo_evaluacion`

Revisemos ahora cómo se implementaría un *pipeline* o serie de *pipelines* en un proyecto. Para esto, es importante entender cómo se estructuran y organizan dentro de un proyecto Kedro, teniendo siempre en cuenta que Kedro adopta el principio de "***pipelines modulares***", para permitir la división de flujos de trabajo complejos en componentes más pequeños y manejables.

En un proyecto Kedro típico, los *pipelines* se organizan siguiendo una estructura de directorios específica:

```
src/
├── <nombre_paquete>/
│   ├── __init__.py
│   ├── pipeline_registry.py # Registro central de todos los pipelines
│   └── pipelines/
│       ├── __init__.py
│       ├── pipeline_datos/ # Un pipeline modular
│       │   ├── __init__.py
│       │   ├── nodes.py    # Definiciones de nodos
│       │   └── pipeline.py  # Definición del pipeline
│       └── pipeline_modelos/ # Otro pipeline modular
│           ├── __init__.py
│           ├── nodes.py
│           └── pipeline.py
```

Figura 10. Estructura de directorios en Kedro.

Esta estructura refleja el principio "un *pipeline*, una carpeta", facilitando la organización y reutilización del código.

La forma recomendada de crear un nuevo *pipeline* es utilizar el comando CLI de Kedro:

```
kedro pipeline create <nombre_pipeline>
```

Este comando genera automáticamente la estructura de carpetas necesaria, incluyendo:

- `__init__.py` para que Python trate el *pipeline* como un módulo.
- `nodes.py` para almacenar las funciones de los nodos.
- `pipeline.py` para definir el *pipeline*.
- Un archivo de parámetros específico del *pipeline* en `conf/base/parameters_<nombre_pipeline>.yaml`.
- Una estructura de pruebas adecuada en `src/tests/pipelines/<nombre_pipeline>/`.

En el siguiente recurso, crearemos un *pipeline* dentro del proyecto de la lección anterior, para el cual, hemos creado un tutorial paso a paso.

Antes de terminar...

En conclusión, los *pipelines* son una manera estructurada de organizar procesos en pasos claros y conectados entre sí. Al representarse como un grafo acíclico dirigido, permiten visualizar mejor el flujo de trabajo, identificar dependencias y dar mayor orden al proyecto. Más allá de la herramienta, lo importante es entender cómo esta lógica facilita la planeación, ejecución y comunicación de procesos complejos.

Ahora, reflexiona sobre lo aprendido:

1. ¿Qué ventajas ofrece el uso de un DAG en comparación con ejecutar las funciones de forma secuencial y manual?
2. ¿Cómo podría ayudarte Kedro a detectar errores en la organización de tus datos o procesos?
3. ¿En qué escenarios de tu propio trabajo o proyectos aplicarías un *pipeline* como el que ofrece Kedro?



Material adicional

Kedro. (s.f.) *A toolbox for production-ready data science* [Una caja de herramientas para ciencia de datos lista para producción]. Kedro. Recuperado el 02 de septiembre de 2025, de: <https://docs.kedro.org/en/1.0.0/index.html>

Kedro. (s.f.) *kedro_datasets*. Kedro. Recuperado el 02 de septiembre de 2025, de: <https://docs.kedro.org/projects/kedro-datasets/en/kedro-datasets-8.1.0/>

Kedro. (s.f.) *How to specify parameters at runtime* [Cómo especificar parámetros en tiempo de ejecución]. Kedro. Recuperado el 02 de septiembre de 2025, de: <https://docs.kedro.org/en/1.0.0/configure/parameters/#how-to-load-parameters-in-code>

Python. (s.f.) *venv – Creation of virtual environments* [venv – Creación de entornos virtuales]. Python. Recuperado el 02 de septiembre de 2025, de: <https://docs.python.org/3/library/venv.html>

Visual studio code. (s.f.) *Tutorial: Get started with Visual Studio Code* [Tutorial: Comienza con Visual Studio Code]. Recuperado el 02 de septiembre de 2025, de: <https://code.visualstudio.com/docs/getstarted/getting-started>

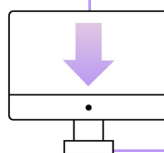
Nota: Los enlaces a las páginas web incluidos en este material adicional fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.

Bibliografía

Kedro. (s.f.) *A toolbox for production-ready data science* [Una caja de herramientas para ciencia de datos lista para producción]. Kedro. Recuperado el 02 de septiembre de 2025, de: <https://docs.kedro.org/en/1.0.0/index.html>

Tech & AI. (06 de junio del 2019). *Meet Kedro, McKinsey's first open-source software tool* [Conoce Kedro, la primera herramienta de software de código abierto de McKinsey]. McKinsey & Company. Recuperado el 02 de septiembre de 2025, de: <https://www.mckinsey.com/about-us/new-at-mckinsey-blog/meet-kedro-mckinseys-first-open-source-software-tool>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 24 de noviembre de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.